

NLP

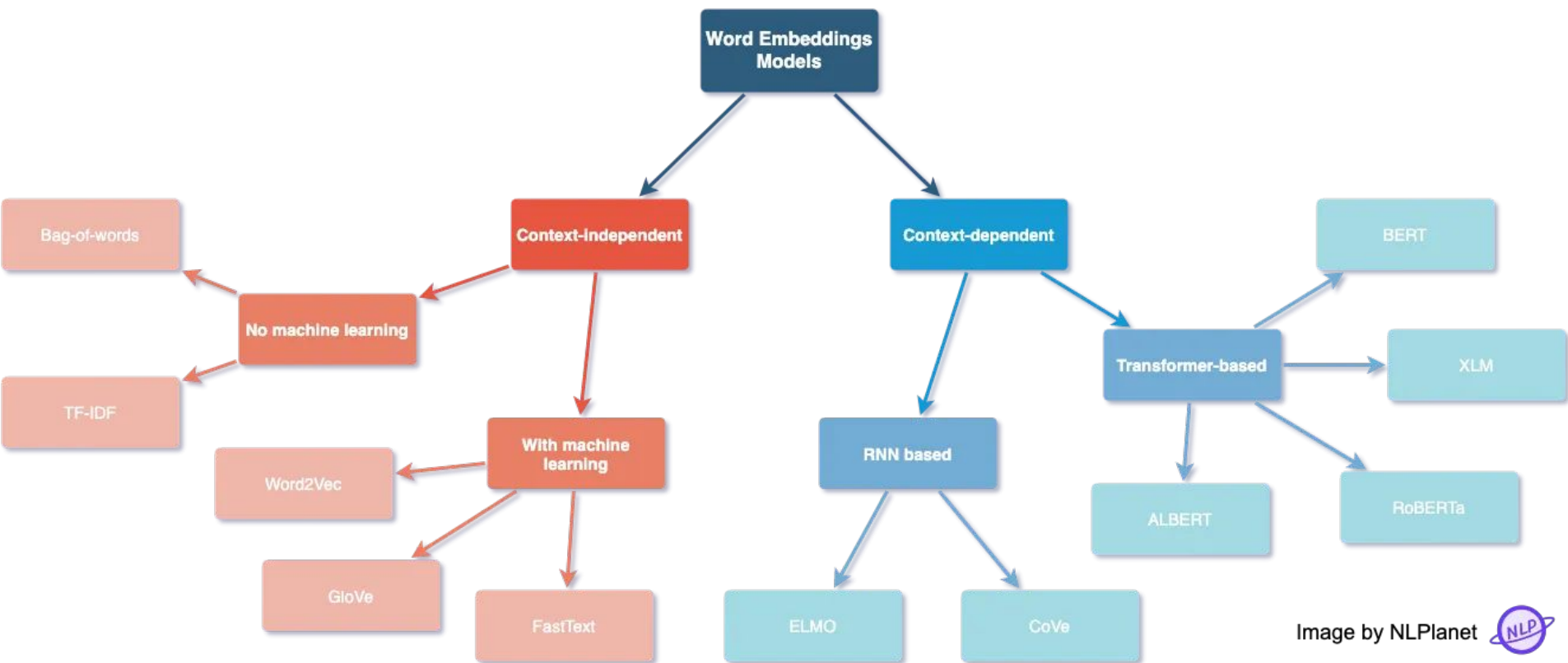
Week#03

Join Team

mww2yo3

Text Representation/Feature Extraction/Text Vectorization

- One hot encoding
- Bag of Words - BoW
- Term Frequency Inverse Term Frequency - TF-IDF
- ngrams
- Word2Vec



One Hot Encoding

One-hot encoding is the process of turning categorical factors into a numerical structure that machine learning algorithms can readily process. **It functions by representing each category in a feature as a binary vector of 1s and 0s, with the vector's size equivalent to the number of potential categories.**

Let's Do an Example

SMS	Label
Win a free iPhone now	spam
Call me when you get this	ham
You have won a price	spam

Drawbacks of One-Hot Encoding in NLP

- Produces high-dimensional sparse vectors
- Leads to increased computational cost and memory usage
- Each word is represented by a unique binary vector, creating a large feature space
- Fails to capture semantic relationships between words
- Models using these vectors may show poor generalization and performance

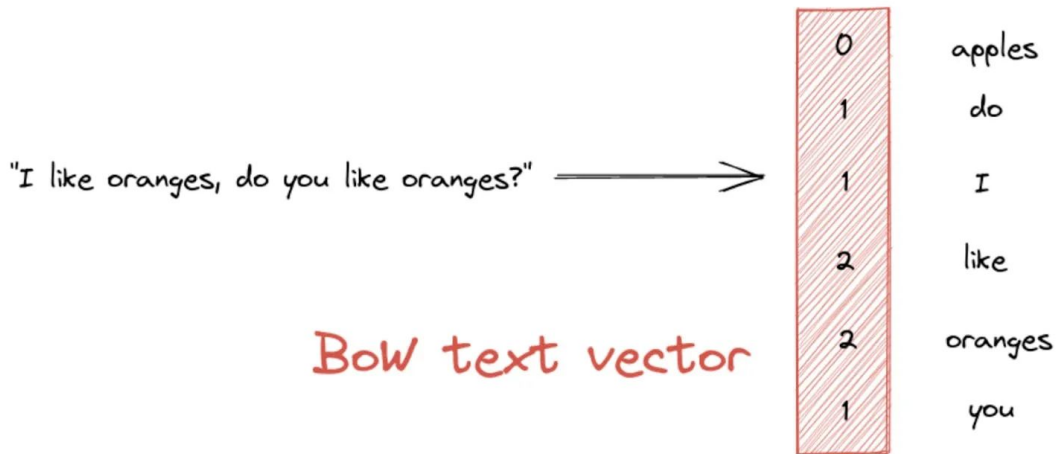
BoW

It turns text like sentence, paragraph or document into a collection of words and counts how often each word appears but ignoring the order of the words. It does not consider the order of the words or their grammar but focuses on counting how often each word appears in the text.

This makes it useful for tasks like text classification, sentiment analysis and clustering.

BoW

One of the simplest vectorization methods for text is a [bag-of-words](#) (BoW) representation. A BoW vector has the length of the entire vocabulary—that is, the set of unique words in the corpus. The vector's values represent the frequency with which each word appears in a given text passage:



Example

SMS	Label
Win win a free iPhone now	spam
Call me when you get this call	ham
You have won a prize prize	spam

Advantages of the Bag of Words Model

- Simplicity: It is easy to implement and computationally efficient.
- Versatility: It can be used for various NLP tasks such as text classification, sentiment analysis and document clustering.
- Interpretability: The resulting vectors are interpretable which makes it easy to understand which words are most important in a document.

Limitations of BoW

1. Loss of Context: It ignores word order and context which means it might miss important relationships between words.
 1. I ate an apple after lunch.”
 2. I bought shares of Apple after lunch.
2. Sparsity: When working with large datasets, most word vectors will be sparse (containing mostly zeros) which can lead to inefficiency.
3. Limited Semantic Understanding: The model doesn't capture the meaning of words which can be important for some NLP tasks.
 1. I am happy today.
 2. I am joyful today.

TF-IDF

TF-IDF is a statistical technique used in NLP and Information Retrieval to determine how important a word is to a document relative to a collection (corpus) of documents.

Term Frequency (TF)

- Measures how often a term appears in a document.
- The more frequent the word, the more important it is within that document.

Formula:

$$\text{TF}(t, d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d}$$

Inverse Document Frequency (IDF)

- Measures how rare or unique a term is across all documents.
- Common words get lower weight, rare words get higher weight.
- Formula:

$$\text{IDF}(t, D) = \log \frac{\text{Total number of documents in corpus } D}{\text{Number of documents containing term } t}$$

Combined TF-IDF Formula

$$\text{TF - IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D)$$

Sent 1: good boy

Sent 2: good girl

Sent 3: boy girl good

$$TF = \frac{\text{No of rep of words in a sentence}}{\text{No of words in a sentence}}$$

$$IDF = \log \left(\frac{\text{No of sentences}}{\text{No of sentences containing word}} \right)$$

	TF		
	S1	S2	S3
good	1/2	1/2	1/3
boy	1/2	0	1/3
girl	0	1/2	1/3

	IDF
good	$\log(3/1) = 0$
boy	$\log(3/2)$
girl	$\log(3/2)$

	TFIDF = TF × IDF		
	good	boy	girl
Sent 1	0	$\frac{1}{2} \times \log\left(\frac{3}{2}\right)$	0
Sent 2	0	0	$\frac{1}{2} \log\left(\frac{3}{2}\right)$
Sent 3	0	$\frac{1}{3} \times \log\left(\frac{3}{2}\right)$	$\frac{1}{3} \log\left(\frac{3}{2}\right)$

$$\frac{1}{2} \log\left(\frac{3}{2}\right) = 0.0880$$

$$\frac{1}{3} \log\left(\frac{3}{2}\right) = 0.058$$

The theoretical value and the actual value is being different by a significant amount.

It is because inside of the scikit library the formula that is being used for IDF is

Press enter or click to view image in full size

- For each term, calculate the inverse document frequency across the entire corpus.
- $IDF(t, D) = \log \left(\frac{\text{Total number of documents in the corpus } N}{\text{Number of documents containing term } t} \right) + 1$

Let's Do a Numerical Example of TF-IDF

The cat sat on the mat
The dog sat on the log
The cat and the dog

N-gram

An N-gram is a sequence of N items (words or characters) from a given text or speech.

They provide a statistical representation of text, helping machines identify language patterns and predict upcoming words.

Types of N-grams

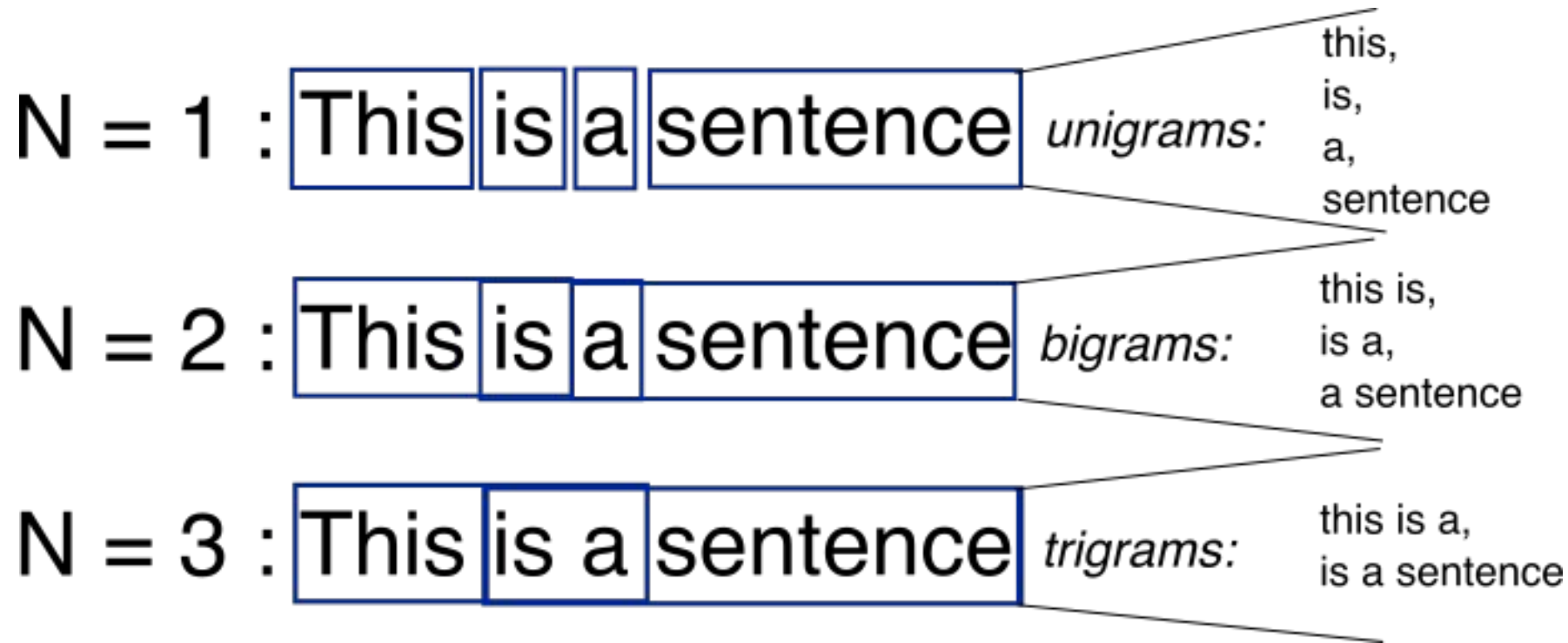
Some of the most common n grams include:

Unigram: Unigrams use single words such as “I”, “want”, and “pizza”. This type of N-gram is helpful for fundamental frequency analysis as you assess the presence of single items and how often they appear in your text. (Note unigram is same as BoW)

Bigram: Bigrams assess pairs of consecutive words, such as “I want” and “want pizza”. This helps to explore how pairs of words relate and how often one word appears after another.

Trigram: Trigrams further analyze sequences of three words, such as “I want pizza”. This helps to provide deeper contextual information about the words and how they relate to each other. You can assign probabilities to the entire sequence of words to better understand how often they appear together.

N-gram: You can go beyond trigrams to assess phrases containing four words, five words, and so on. You can determine the appropriate N-gram based on your task and data set. The functionality will remain the same, except you will compute the probability of words appearing in larger sequences.



Example of N-Grams

I live in Pakistan

Type of n gram	Generated n-grams
Unigram	
Bigram	
Trigram	

Example of N-Grams

I live in Pakistan

Type of n gram	Generated n-grams
Unigram	["I", "live", "in", "Pakistan"]
Bigram	["I live", "live in", "in Pakistan"]
Trigram	["I live in", "live in Pakistan"]

Example

Doc 1 = This smartwatch is very good.

Doc 2 = This smartwatch is not good.

	This	smartwatch	is	very	not	good
Doc 1	1	1	1	1	0	1
Doc 2	1	1	1	0	1	1

Bigram of this example

Doc 1 = This smartwatch is very good.

Doc 2 = This smartwatch is not good.

Example

Coding Example

https://colab.research.google.com/drive/1X4kb8IM0D1M_buMbJ6PdKNGiCpaPiOVO?usp=sharing

Quiz

Solve the questions provided in the sheet